

PHP MySQL

Database

MySQL is a database system used on the web. It runs on a server and is ideal for both small and large applications. MySQL is used for a wide range of purposes, including data warehousing, e-commerce, and logging applications. The most common use for MySQL, however, is for a web database. It can be used to store anything from a single record of information to an entire inventory of available products for an online store.

In association with a scripting language such as PHP, it is possible to create websites that will interact in real-time with a MySQL database to rapidly display categorized and searchable information to a website user.

MySQL database server offers several advantages:

- MySQL is easy to use, yet extremely powerful, fast, secure, and scalable.
- MySQL runs on a wide range of operating systems, including UNIX or Linux, Microsoft Windows, Apple Mac OS X, and others.
- MySQL supports standard SQL (Structured Query Language).
- MySQL is an ideal database solution for both small and large applications.
- MySQL is developed and distributed by Oracle Corporation.
- MySQL includes data security layers that protect sensitive data from intruders.

MySQL Connection

In the context of database programming, a **connection** is like a pipeline of sorts that allows communication between a DBMS and an application program. With MySQL databases, we have to supply the following information when making a database connection: the host or URL of the database server, the database name, and the database user name and password.

Opening Database Connection

In PHP you can easily do this using the `mysqli_connect()` function. All communication between PHP and the MySQL database server takes place through this connection. Here're the basic syntax for connecting to MySQL:

Syntax: `MySQLi`

```
$link = mysqli_connect("hostname", "username", "password", "database");
```

The hostname parameter in the above syntax specify the hostname (e.g., localhost) or IP address of the MySQL server, whereas the username and password parameters specify the credentials to access the MySQL server, and the database parameter, if provided, will specify the default MySQL database to be used when performing queries.

The following example shows how to connect to MySQL database server using MySQLi:

```
<?php
$servername = "localhost";
$username = "username";
$password = "password123";

// Create connection
```

```
$conn = mysqli_connect($servername, $username, $password);

// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
?>
```

Closing Database Connection

The connection will be closed automatically when the script ends. To close the connection before, use the following:

```
mysqli_close($conn);
```

Create Database and Table

The **CREATE DATABASE** statement is used to create a database in MySQL. To create and delete a database, you should have admin privilege. PHP uses the **mysql_query** function to create a MySQL database. This function takes two parameters and returns TRUE on success or FALSE on failure.

The following example create a database named "myDB":

```
<?php
$servername = "localhost";
$username = "username";
$password = "password123";

// Create connection
$conn = mysqli_connect($servername, $username, $password);
// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

// Create database
$sql = "CREATE DATABASE myDB";
if (mysqli_query($conn, $sql)) {
    echo "Database created successfully";
} else {
    echo "Error creating database: " . mysqli_error($conn);
}

mysqli_close($conn);
?>
```

Note: When you create a new database, you must only specify the first three arguments to the *mysqli* object (*servername*, *username*, and *password*).

Create Table

A database table has its unique name and consists of columns and rows. The CREATE TABLE statement is used to create a table in MySQL. To create tables in the new database, you need to do the same thing as creating the database.

We will create a table named "MyGuests", with five columns: "id", "firstname", "lastname", "email" and "reg_date":

```
CREATE TABLE MyGuests (  
  id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  firstname VARCHAR(20) NOT NULL,  
  lastname VARCHAR(20) NOT NULL,  
  email VARCHAR(40),  
  reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
)
```

Notes on the table above:

The data type specifies what type of data the column can hold. After the data type, you can specify other optional attributes for each column:

- **NOT NULL** – Each row must contain a value for that column; null values are not allowed
- **DEFAULT VALUE** – Set a default value that is added when no other value is passed
- **UNSIGNED** – Used for number types, limits the stored data to positive numbers and zero
- **AUTO INCREMENT** – MySQL automatically increases the value of the field by 1 each time a new record is added
- **PRIMARY KEY** - Used to uniquely identify the rows in a table. The column with the PRIMARY KEY setting is often an ID number and is often used with AUTO_INCREMENT.

Each table should have a primary key column (in this case: the "id" column). Its value must be unique for each record in the table. The following example shows how to create the table in PHP:

```
<?php  
$servername = "localhost";  
$username = "username";  
$password = "password123";  
$dbname = "myDB";  
  
// Create connection  
$conn = mysqli_connect($servername, $username, $password, $dbname);  
// Check connection  
if (!$conn) {  
    die("Connection failed: " . mysqli_connect_error());  
}  
  
// sql to create table  
$sql = "CREATE TABLE MyGuests (  
  id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  firstname VARCHAR(20) NOT NULL,  
  lastname VARCHAR(20) NOT NULL,  
  email VARCHAR(40),  
  reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
```

```
);  
  
if (mysqli_query($conn, $sql)) {  
    echo "Table MyGuests created successfully";  
} else {  
    echo "Error creating table: " . mysqli_error($conn);  
}  
  
mysqli_close($conn);  
?>
```

Select, Insert, Delete, and Update Data

Select Data from a MySQL Database

The SELECT statement is used to select data from one or more tables:

```
SELECT column_name(s) FROM table_name
```

or we can use the * character to select ALL columns from a table:

```
SELECT * FROM table_name
```

The following example selects the *id*, *firstname*, and *lastname* from the *MyGuests* table and displays it on the page:

```
<?php  
$servername = "localhost";  
$username = "username";  
$password = "password123";  
$dbname = "myDB";  
  
// Create connection  
$conn = mysqli_connect($servername, $username, $password, $dbname);  
// Check connection  
if (!$conn) {  
    die("Connection failed: " . mysqli_connect_error());  
}  
  
$sql = "SELECT id, firstname, lastname FROM MyGuests";  
$result = mysqli_query($conn, $sql);  
  
if (mysqli_num_rows($result) > 0) {  
    // output data of each row  
    while($row = mysqli_fetch_assoc($result)) {  
        echo "id: " . $row["id"]. " - Name: " . $row["firstname"]. " " . $row["lastname"]. "<br>";  
    }  
} else {  
    echo "0 results";  
}  
  
mysqli_close($conn);  
?>
```

Code lines to explain from the example above:

First, we set up an SQL query that selects the *id*, *firstname*, and *lastname* columns from the *MyGuests* table. The next line of code runs the query and puts the resulting data into a variable called `$result`. Then, the function `num_rows()` checks if there are more than zero rows returned.

If there are more than zero rows returned, the function `fetch_assoc()` puts all the results into an associative array that we can loop through. The `while()` loop loops through the result set and outputs the data from the *id*, *firstname*, and *lastname* columns.

Insert Data into MySQL

After a database and a table have been created, you can start adding data to them. Here are some syntax rules to follow:

- The SQL query must be quoted in PHP
- String values inside the SQL query must be quoted
- Numeric values must not be quoted
- The word NULL must not be quoted

The INSERT INTO statement is used to add new records to a MySQL table:

```
INSERT INTO table_name (column1, column2, column3,...)
VALUES (value1, value2, value3, ...)
```

The following example adds a new record to the *MyGuests* table:

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = mysqli_connect($servername, $username, $password, $dbname);
// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('Peter', 'Parker', 'peterparker@mail.com')";

if (mysqli_query($conn, $sql)) {
    echo "New record created successfully";
} else {
    echo "Error: " . $sql . "<br>" . mysqli_error($conn);
}

mysqli_close($conn);
?>
```

Delete Data from a MySQL Table

The DELETE statement is used to delete records from a table:

```
DELETE FROM table_name  
WHERE some_column = some_value
```

Note: The **WHERE** clause specifies which record or records that should be deleted. If you forget the WHERE clause, all records will be deleted.

Let's look at the *MyGuests* table:

id	firstname	lastname	email	reg_date
1	Peter	Parkor	peterparker@mail.com	2021-10-26 06:13:07
2	Tony	Starks	tonystarks@mail.com	2021-10-27 10:13:25
3	Natasha	Romanoff	natasharomanoff@mail.com	2021-10-29 12:30:25

The following example deletes the record with id=2 in the *MyGuests* table:

```
<?php  
$servername = "localhost";  
$username = "username";  
$password = "password123";  
$dbname = "myDB";  
  
// Create connection  
$conn = mysqli_connect($servername, $username, $password, $dbname);  
// Check connection  
if (!$conn) {  
    die("Connection failed: " . mysqli_connect_error());  
}  
  
// sql to delete a record  
$sql = "DELETE FROM MyGuests WHERE id=2";  
  
if (mysqli_query($conn, $sql)) {  
    echo "Record deleted successfully";  
} else {  
    echo "Error deleting record: " . mysqli_error($conn);  
}  
  
mysqli_close($conn);  
?>
```

After the record is deleted, the table will look like this:

id	firstname	lastname	email	reg_date
1	Peter	Parkor	peterparker@mail.com	2021-10-26 06:13:07
3	Natasha	Romanoff	natasharomanoff@mail.com	2021-10-29 12:30:25

Update Data in a MySQL Table

The UPDATE statement is used to update existing records in a table:

```
UPDATE table_name  
SET column1=value, column=value2, ...  
WHERE some_column=some_value
```

Let's look at the *MyGuests* table:

id	firstname	lastname	email	reg_date
1	Peter	Parkor	peterparker@mail.com	2021-10-26 06:13:07
3	Natasha	Romanoff	natasharomanoff@mail.com	2021-10-29 12:30:25

The following examples update the record with id=1 in the *MyGuests* table:

```
<?php  
$servername = "localhost";  
$username = "username";  
$password = "password";  
$dbname = "myDB";  
  
// Create connection  
$conn = mysqli_connect($servername, $username, $password, $dbname);  
// Check connection  
if (!$conn) {  
    die("Connection failed: " . mysqli_connect_error());  
}  
  
$sql = "UPDATE MyGuests SET lastname='Parker' WHERE id=1";  
  
if (mysqli_query($conn, $sql)) {  
    echo "Record updated successfully";  
} else {  
    echo "Error updating record: " . mysqli_error($conn);  
}  
  
mysqli_close($conn);  
?>
```

After the record is updated, the table will look like this:

id	firstname	lastname	email	reg_date
1	Peter	Parker	peterparker@mail.com	2021-10-26 06:13:07
3	Natasha	Romanoff	natasharomanoff@mail.com	2021-10-29 12:30:25

REFERENCES:

Aley, R. (2016). *PHP beyond the web*. Apress Media, LLC.

Bramer, M. (2015). *Web programming with PHP and MySQL: A practical guide*. Springer International Publishing.

Connolly, R. & Hoar, R. (2015). *Fundamentals of web development*. Pearson Education, Inc.